RAJALAKSHMI
ENGINEERING COLLEGE

An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

BONAFIDE CERTIFICATE

NAME ....AISHWARYA.B.....

ACADEMIC YEAR .2026-27... SEMESTER ....V.....BRANCH.....CSD.....

UNIVERSITY REGISTER No. 211624170100 3

Certified that this is the bonafide record of work done by the above student in the

IT23E31 Graphics Laboratory during the year 2026 - 2027
and Multimedia

Signature of Faculty - in - Charge

Submitted for the Practical Examination held on ....03/11/2026.....

Internal Examiner

External Examiner

Aishwarya B
CSD

241701003

INDEX

NAME _____ SUBJECT _____

STD. _____ DIV. _____ ROLL NO. _____ SCHOOL _____

SR. NO.	PAGE NO.	TITLE	DATE	TEACHER'S SIGN / REMARKS
1	17/7	Bresenhen line drawing		
2	17/7	Bresenhem line drawing - circle		
3	17/7	Bresenhen line algorithm - ellipse		
4	24/7	Drawing butterfy using midpoint algorithm		
5	24/7	Drawing smiley face using midpoint circle algorithm		
6	31/7	Flood fill algorithm		
7	7/8	Olympic logo using elipse.		
8	14/8	Basic Transformation for 2D Shapes for Triangles		
9	14/8	Basic Transformation for 2D shapes for Rechangle		
10	21/8	Sutherland Hodgman Polygon Clipping Algorithm		
11	11/9	3D Shapes transformarion for Cube		
12	18/9	3D Shapes for Cone		
13	25/9	Rendering 3D objects using polygons and Shading techniques		
14	5/10	Multimedia Application interfaces.		
15	9/10	Capture audio/vide from webcam and microphone and display multimedia		

Completed 4/10/26

Exp1

17/7/26

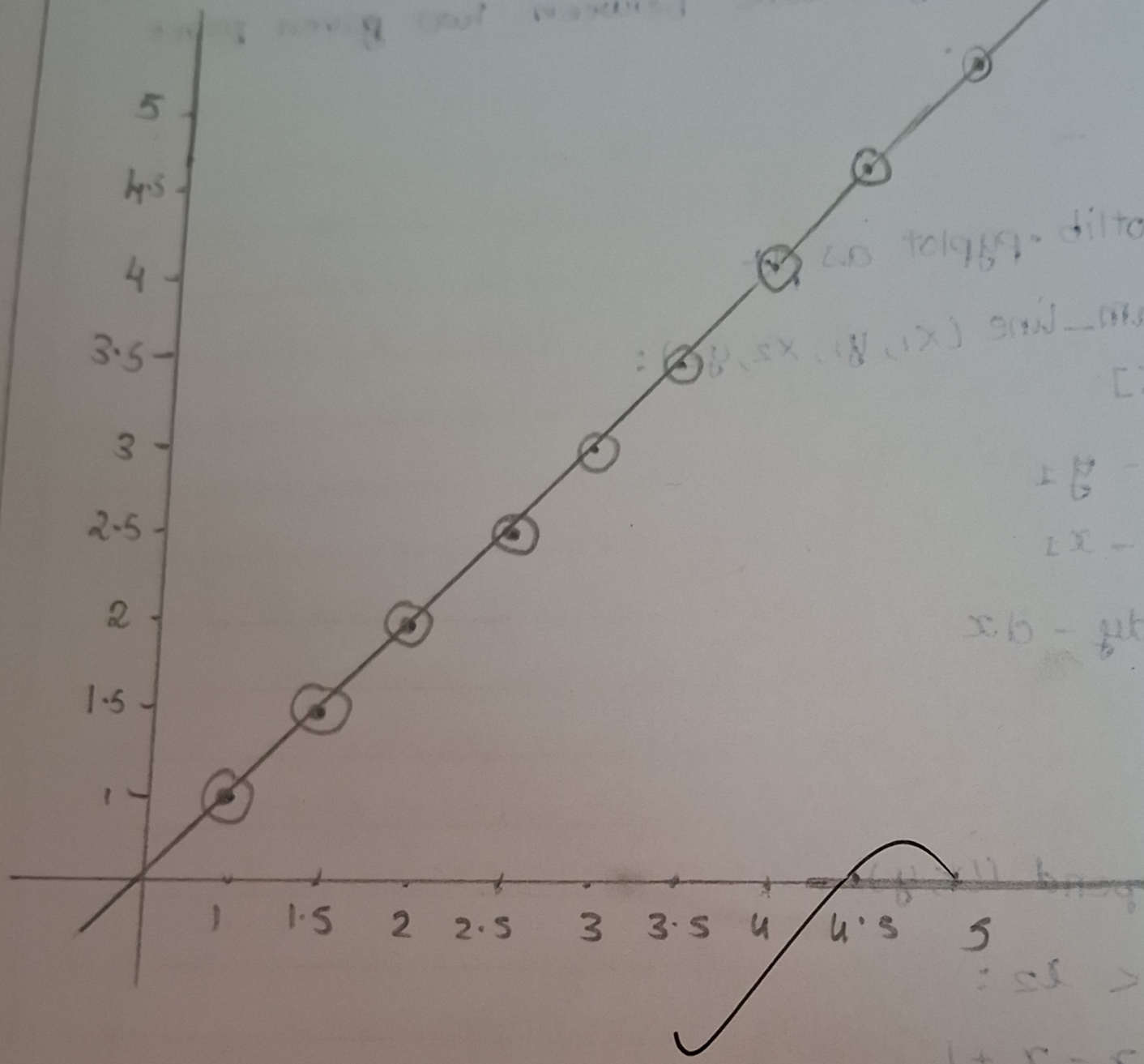
BRESENHAM LINE DIAGRAM

AIM:-

To implement Bresenham's Line Drawing Algorithm to genrate and display a straight line between two given point

CODING:

```
import matplotlib.pyplot as plt
def bresenham_line (x1, y1, x2, y2):
    points = []
    dx = x2 - y1
    dy = y2 - x1
    P = 2 * dy - dx
    x = x1
    y = y1
    point.append ((x, y))
    while x < x2:
        x = x + 1
        if p < 0:
            p = p + 2 * dy
        else:
            y = y + 1
            p = p + 2 * dy - 2 * dx
        point.append ((x, y))
    return points
x1 = int (input ("Enter x1: "))
y1 = int ( input ("Enter y1:"))
x2 = int ( input ("Enter x2:"))
y2 = int (input ("Enter y2:"))
points = bresenham_line (x1, y1, x2, y2)
print ("Point genrated:")
```


5
4.5
4
3.5
3
2.5
2
1.5
1
1
1.5
2
2.5
3
3.5
4
4.5
5

```
for p in points:
    print(p)

x_value = [p[0] for p in points]
y - value = [p[1] for p in points]
plt.plot (x_value, y-value, marker = "o")
plt.title ("Bresenham Line drawing Algorithm (|m| <1)")
plt.xlabel ("x_axis")
plt.ylabel ("y_axis")
plt.grid ()
plt.show()
```

RESULT:

The brensenham line drawing was successfully executed.

Exp 2

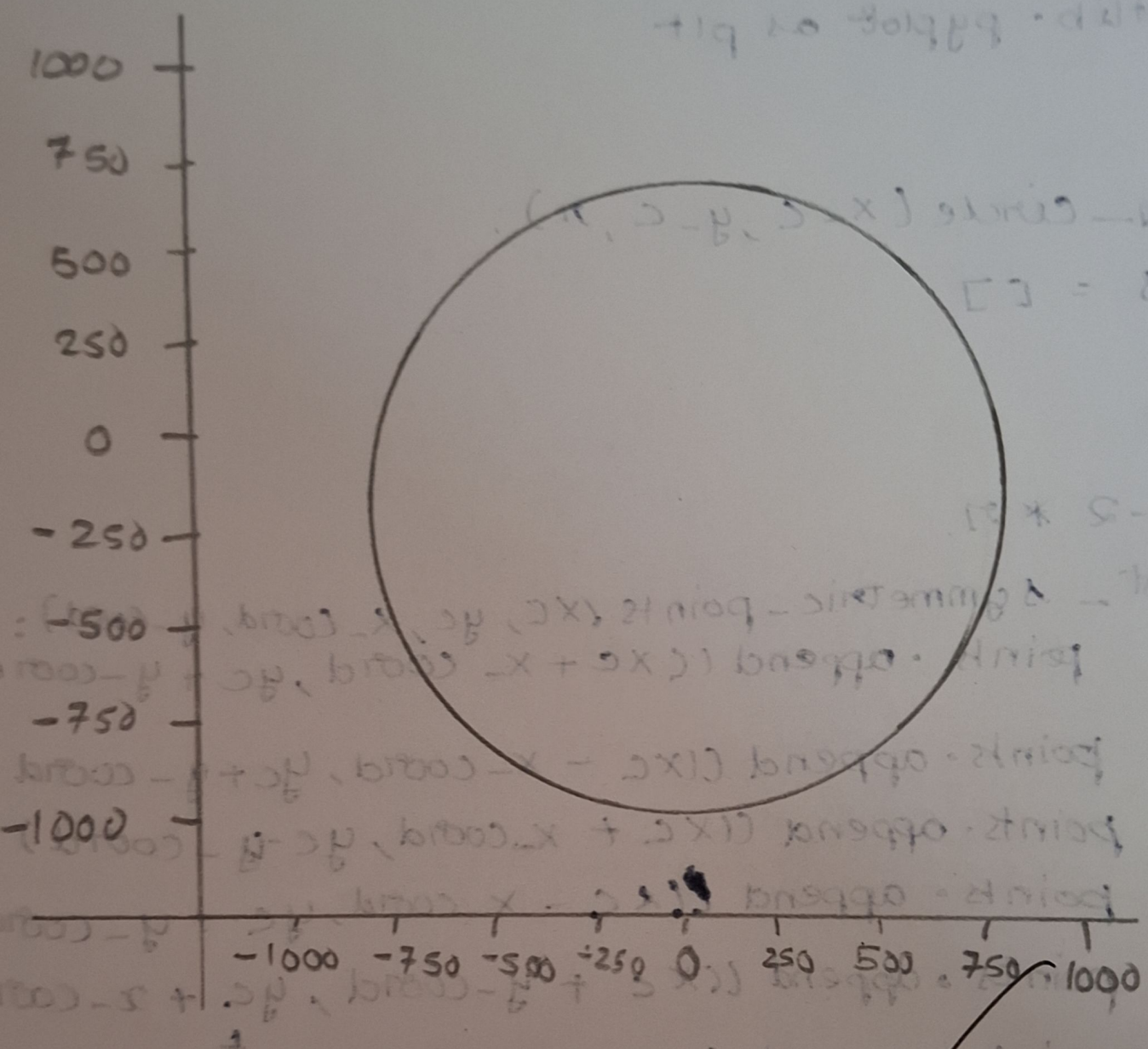
17/7/26 BRESENHAM CIRCLE DIAGRAM

AIM:

To implement bresenham circle diagram to generate and display circle

CODE:

```
import matplotlib.pyplot as plt
import math
def bresenham_circle(x_c, y_c, r):
    points = []
    x = 0
    y = r
    d = 3 - 2 * r
    def plot_symmetric_points(xc, yc, x_coord, y_coord):
        points.append((xc + x_coord, yc + y_coord))
        points.append((xc - x_coord, yc + y_coord))
        points.append((xc + x_coord, yc - y_coord))
        points.append((xc - x_coord, yc - y_coord))
        points.append((xc + y_coord, yc + x_coord))
        points.append((xc - y_coord, yc + x_coord))
        points.append((xc + y_coord, yc - x_coord))
        points.append((xc - y_coord, yc - x_coord))
    plot_symmetric_points(x_c, y_c, x, y)
    while y >= x:
        x += 1
        if d > 0:
            y -= 1
            d = d + 4 * (x - y) + 10
        else:
            d = d + 4 * x + 6
        plot_symmetric_points(x_c, y_c, x, y)
    return points
```


1000
750
500
250
0
-250
-500
-750
-1000
-1000 -750 -500 -250 0 250 500 750 1000

```
x_center = int(input("Enter the x-coordinate of the center: "))
y_center = int(input("Enter the y-coordinate of the center: "))
radius = int(input("Enter the radius of the circle: "))

raw_circle_points = bresenham_circle(x_center, y_center, radius)
unique_points = list(set(raw_circle_points))
def get_angle(point):
    px, py = point
    return math.atan2(py - y_center, px - x_center)
sorted_points = sorted(unique_points, key=get_angle)
x_coords = [p[0] for p in sorted_points]
y_coords = [p[1] for p in sorted_points]

if sorted_points:
    x_coords.append(sorted_points[0][0])
    y_coords.append(sorted_points[0][1])

plt.figure(figsize=(6,6))
plt.plot(x_coords, y_coords, '-')
plt.title("Bresenham Circle Drawing Algorithm")
plt.xlabel("x-axis")
plt.ylabel("y-axis")
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

RESULT:

The bresenham circle drawing was successfully executed.

Exp: 3

17/7/26

BRESENHAM ELLIPSE DIAGRAM

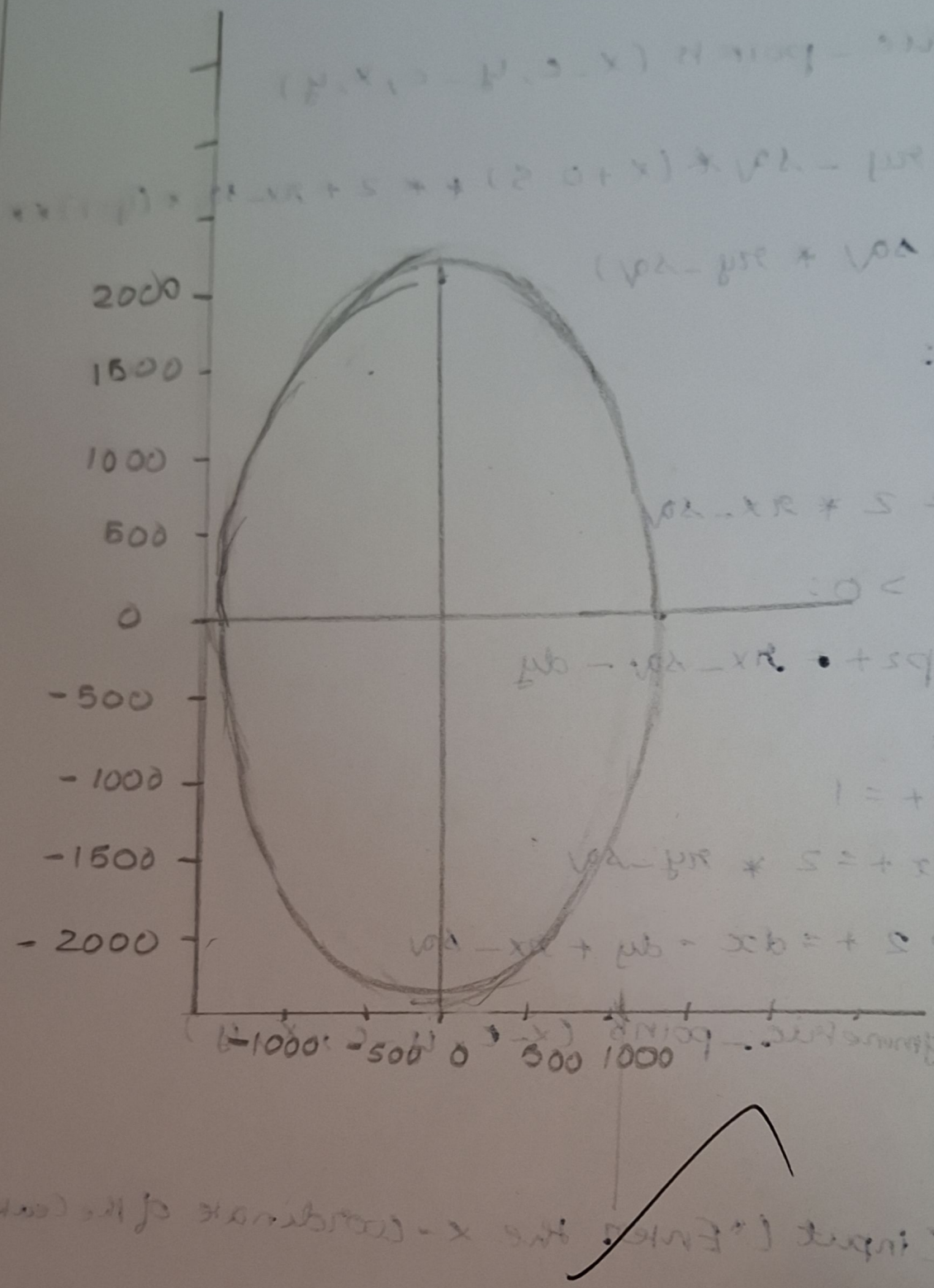
AIM:

To implement bresenham ellipse diagram to generate and display ellipse

CODE

```
import matplotlib.pyplot as plt
import math
def bresenham_ellipse (x_c, y_c, rx, ry):
    points = []
def plot_symmetric_points (xc, yc, x, y):
    points.append ((xc + x, yc + y))
    points.append ((xc - x, yc + y))
    points.append ((xc + x, yc - y))
    points.append ((xc - x, yc - y))
rx_sq = rx * rx
ry_sq = ry * ry
x = 0
y = ry
p1 = round (ry_sq - (rx_sq * ry) + (0.25 * rx_sq))
dx = 2 * ry_sq * x
dy = 2 * rx_sq * y
plot_symmetric_points (x_c, y_c, x, y)
while dx < dy:
    x += 1
    dx += 2 * ry_sq
    if p1 < 0:
        p1 += dx + ry_sq
```

```
else:
        y_ -= 1
        dy_ -= 2 * rx_sq
        p1 += dx_ - dy + ry_sq

    plot_symmetric_points (x_c, y_c, x, y)

p2 = round ( ry_sq * (x + 0.5) ** 2 + rx_sq * (y-1) ** 2
        - rx_sq * ry_sq)

while y >= 0:
        y_ -= 1
        dy_ -= 2 * rx_sq
        if p2 > 0:
            p2 += rx_sq - dy
        else:
            x += 1
            dx += 2 * ry_sq
            p2 += dx - dy + rx_sq

    plot_symmetric_points (x_c, y_c, x, y)

return points

x_center = int (input ("Enter the x-coordinate of the center:"))
y_center = int (input ("Enter the y- coordinate of the center:"))
radius_x = int (input ("Enter the x- radius (semi-major/minor axis):"))
radius_y = int (input ("Enter the y_ radius ("semi-major/minor axis:"))
raw_ellips_points = bresenham_ellipse (x_center, y_center
    radius_x, radius_y)

uniaue_points = list (set (raw_ellipse_points))
def get_angle (radius):
        px, py = rddius
        return math.tan2 (py - y_center, px - x_center)
```


2000
1500
1000
500
0
-500
-1000
-1500
-2000
-1000
-500
0
500
1000

```
sorted_points = sorted(unique_points, key=get_angle)

plt.figure(figsize=(6,6))
plt.plot(x_coords, y_coords, '-')
plt.title("Bresenham Ellipse drawing Algorithm")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.grid(True)
plt.gca().set_aspect('equal', adjustable='box')
plt.show()
```

RESULT:

Thus the bresenham ellipse drawing was successfully implemented.

Exp 3
24/7/26 DRAW BUTTERFLY USING MIDPOINT ALGORITHM

AIM:

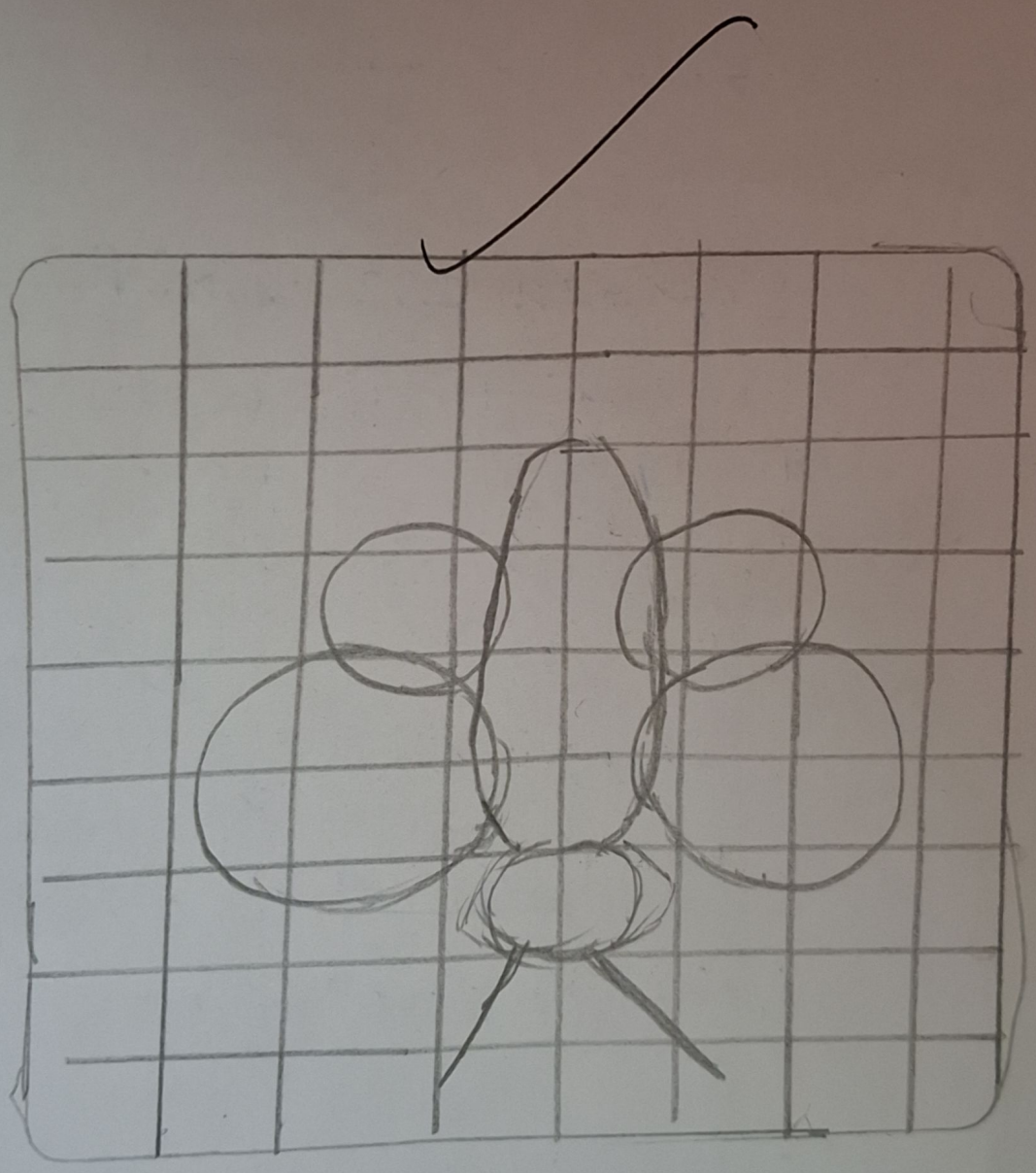
To draw a butterfly using midpoint algorithm

CODE:

```
import numpy as np, matplotlib as plt
def get_circle (xc, yc, n)
while x <= y:
        for dx, dy in
                pts.append (xc, dx, yc + dy)
                p, y = p + 2 * x + 3 * y)
                x += 1
    return pts
def plot_fill (pts, xc, yc, color)
    plt = sorted (set (pts), key = lambda
    p: np.arctan2 (p[1] - yc, p[0] - xc))
    plt.figure (figsize (6,6))
    plot_fill (get_ellipse (2, 6, 4, 3) 'hot pink')
    plot_fill (get_circle (5, 6, 1) 'black')
    plot_fill ('get_circle 'black')
    plt.axis ('equal')
    plt.grid (True)
    plt.show
```

RESULT:

Thus a butterfly is drawn successfully.



Exp 6

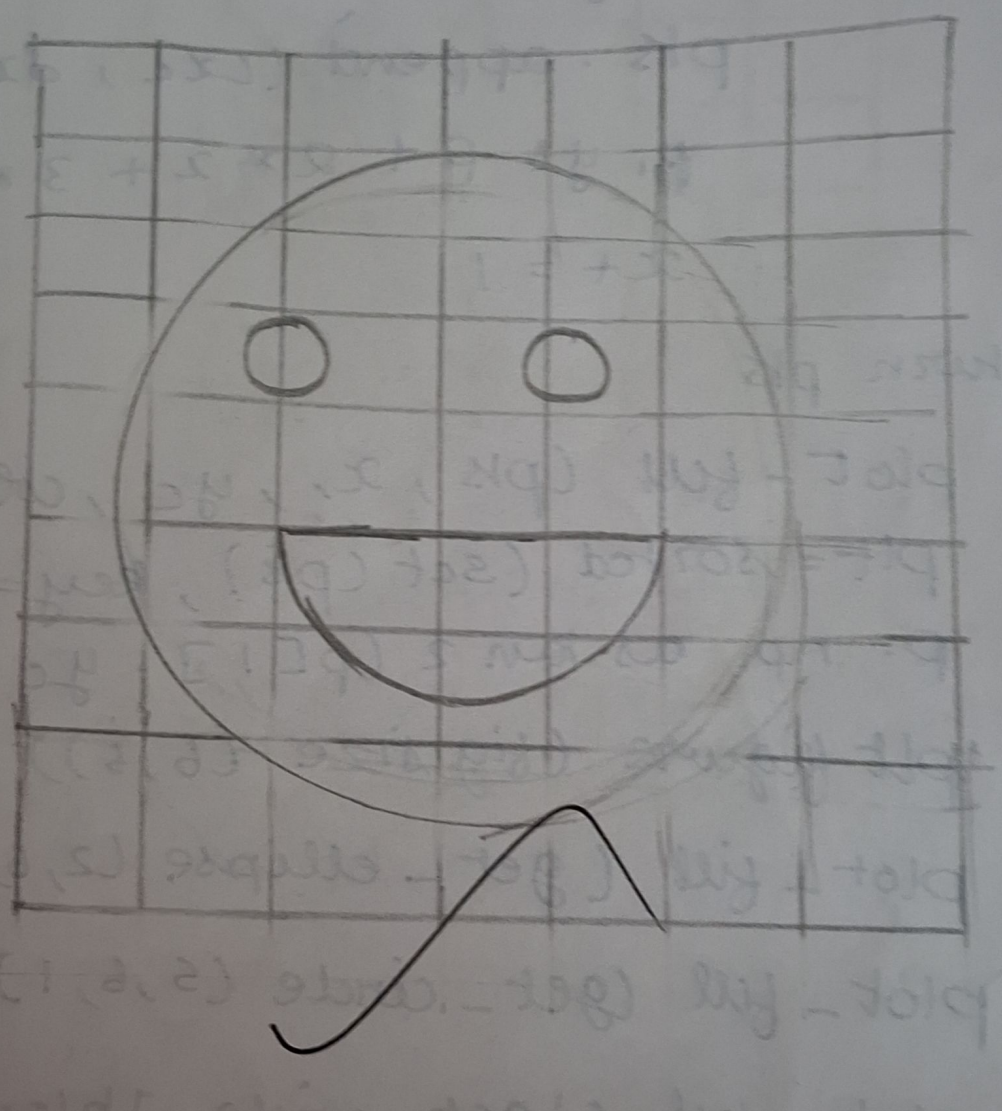
24/7/26 DRAW SMILEY FACE USING MIDPOINT CIRCLE ALGORITHM

AIM:

To draw a smiley face using midpoint circle algorithm

CODE:

```
import matplotlib.pyplot as plt
import numpy as np

def midpoint_circle(xc, yc, r)
    point = []
    x = 0
    y = r
    p = 1 - r
    while x <= y:
        points.extend([
            (xc + x, yc + y),
            (xc - x, yc + y),
            (xc + x, yc - y)
            (xc - x, yc - y)
            (xc + y, yc + x)
            (xc - y, yc + x)
            (xc + y, yc - x)
            (xc - y, yc - x)

    if p < 0:
        p += 2 * x + 3
    else:
        p += 2 * (x - 2) + 5
        y -= 1
    x += 1

return points.
```



```
def filled_circle (xc, yc, r, color
    y= np.arrance (yc - r, yc+r, 0.01)
    for yy in y:
        half = np.sqrt (r* r - (yy- yc)**2)
        plt.plot (
            [xc - half, xc+ half];
            [yy, yy]
            color = color
)
t=np.linspace (0, 2* np.pi, 300)
x= xc + r* np.cos (t)
y= yc + r * np.sin (t)
plt.plot (x, y, color = "black", lenwidth = 5)
filled_circle (0, 0, 5, "yellow")
filled_circle (0 - 1.8, 1.5, 0.5, "black")
filled_circle (0 - 1.8, 1.5, 0.5, "black")
x= 2.5* np.cos(t)
y= 1.5*pp.sin(t)-0.5
plt.show()
```

RESULT:

To draw smiley face using midpoint algorithm is drawn successfully.

Exp 6

31/7/26

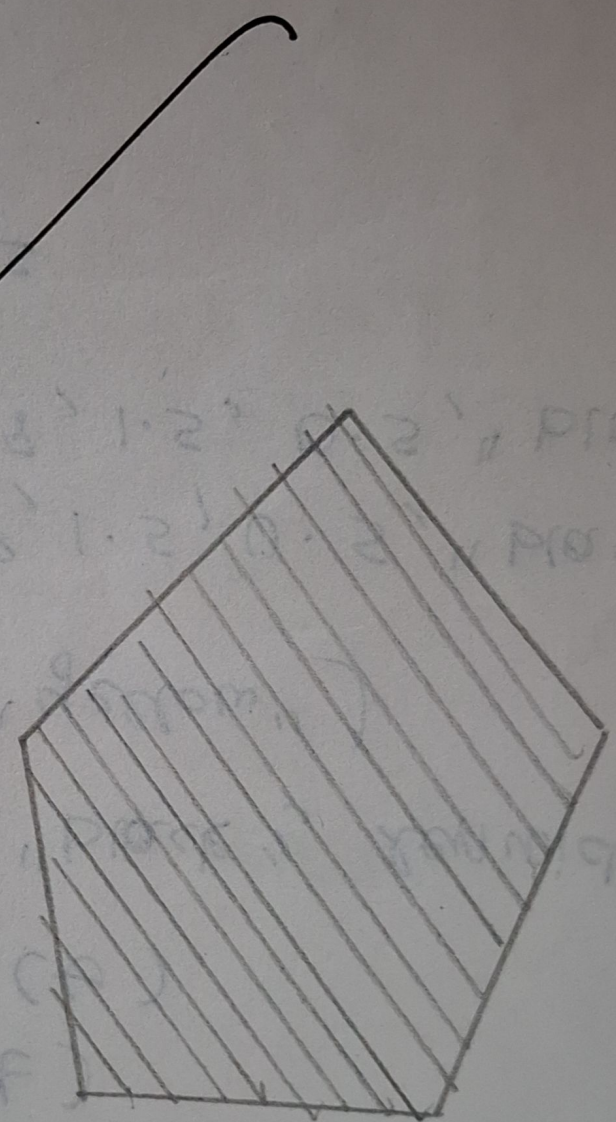
FLOOD FILL ALGORITHM

AIM:

To draw a polygon using flood fill algorithm

CODE:

```
import matplotlib.pyplot as plt
import numpy as plt
width = 300
height = 300
image = np.ones(heigh, width, 3), dtype = np.unit8)*255
Polygon = [(150, 150) (250, 130), (210, 240), (90, 240),
          (50, 130)]
def draw_line (x1, y1, x2, y2):
        points = max (abs (x2-x1), abs (y2-y1)
        for i in range (points + 1)
                x = int (x1 + (x2-x1) * i/points)
                y = int (y1 + (y2-y1) * i/points)
                image [y, x] = [0, 0, 0]
for i in range (len (polygon)):
        x1, y1 = polygon [i]
        x2, y2 = polygon [(i+1) % len (polygon)]
        draw-line (x1, y1, x2, y2)
def flood_fill (x, y, old_color, new-color):
        stack = [(x, y)]
        while stack:
                x, y = stack.pop()
                if (x<0) or x >= width or
y<0 or y >= height
                continue.
        if not np array_equal (image [y, x]
old, color):
        Continue.
```



```
        image [y, x] = new_color
        stack.append ((x+1, y))
        stack.append ((x-1, y))
        stack.append((x, y+1))
        stack.append((x, y-1))

flood_fill(
        150,
        150,
        np.array ([255, 255, 255], dtype = np.uint8)
        np.array ([0, 180, 0], dtype = np.uint8)
)

plt.imshow (image)
plt.title ("Flood fill algorithm)
plt.axis ('off')
plt.show()
```

RESULT:

To draw the polygon using flood fill algorithm is successfully drawn.

Exp 7

7/8/24 OLYMPIC LOGO USING ELLIPSE

AIM:

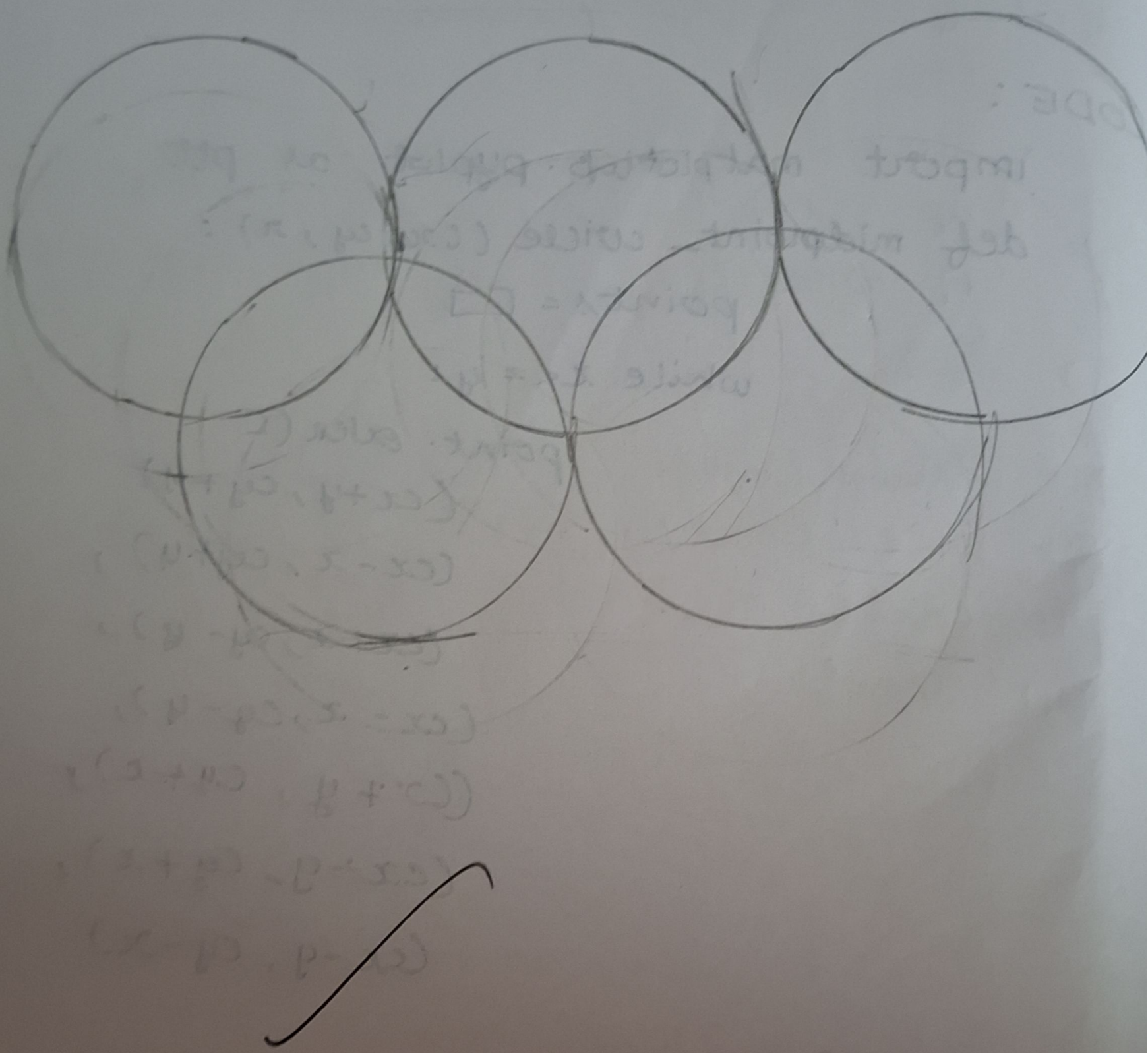
To draw the olympic logo using elipse and mid-point algorithm.

CODE:

```
import matplotlib.pyplot as ptt
def midpoint_circle(cx, cy, r):
    points = []
    while x<=y:
        point.extend([
            (cx+y, cy+y)
            (cx-x, cy+y),
            (cx+x, cy-y),
            (cx-x, cy-y),
            (cx+y, cy+x),
            (cx-y, cy+x),
            (cx-y, cy-x)
```

I)

```
if p<0:
    p= p+2*x+3
else
    p= p+2*(x-y)+7
    y= 1
x+=1
return points.
```



```
def draw_circle (cx, cy, r):
    point = midpoint_circle (cx, cy, r)
    points.sort (key. lambda p: - import..)
    ('math'). atan 2 (p[1]-cy, p[0]-cx))
    x=[p[0] for p in points]
    y=[p[1] for p in points]
    plt.points (x, y, linewidth := 3)

draw_circle (0, 60, r)
draw_circle (110, 60, r)
draw_circle (220, 60, r)
draw_circle (55, 0, r)
draw_circle (165, 0, r)
plt.axis ("equal")
plt.axis ("off")
plt.show ()
```

RESULT:

The olympic logo using midpoint is drawn.

12/8/26 BASIC TRANSFORMATIONS FOR 2D SHAPES FOR TRIANGLE EXP-8

AIM:

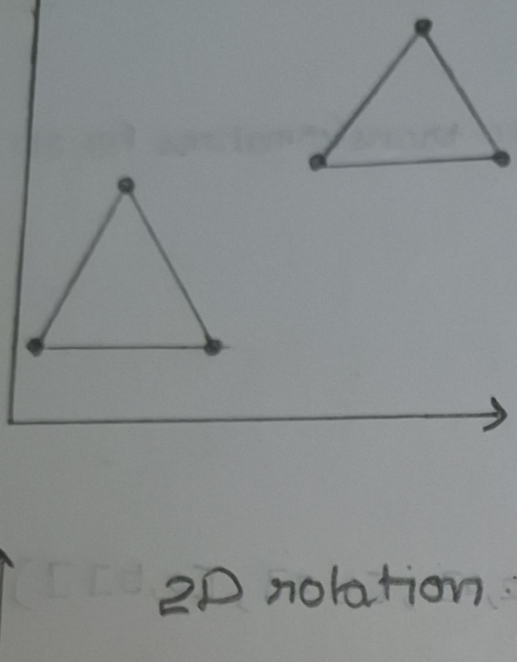
To implement the project for basic transformations for 2D shapes to draw triangle in python.

PROGRAM:

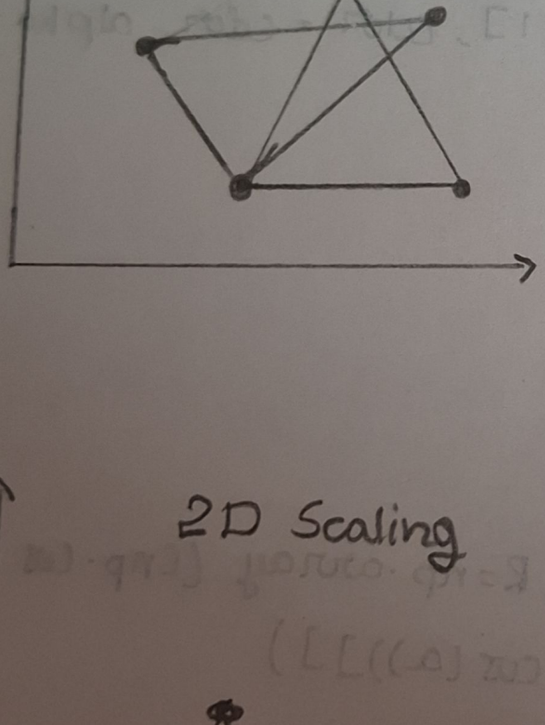
```
import numpy as np
import matplotlib.pyplot as plt
shape = np.array([[0,0],[2,0],[1,2],[0,0]])
def plot(ax, s, color, label):
    ax.plot(s[:,0], s[:,1], color=color, marker='o', label)
    ax.fill(s[1,0], s[s:I, 1], color = color, alpha=0.3)
    ax.grid()
    ax.axis('equal')
    ax.legend()

def translate(s, dx, dy):
    return s + [dx, dy]
def rotate(s, angle):
    a=np.radius(angle), R=np.array([np.cos(a) - np.
sin(a)], [np.sin(a), np.cos(a))]])
return s@R,T
def scale(s, sx, sy)
    return s*(sx, sy)
i) t = translate(shape, 2, 1)
   fig, ax = plt.subplots()
   plot(ax, shape, 'blue', 'Original'), plot(ax, t, 'red'
'Translate' ax.set_title('2D translation)
   plt.show()
ii) r = rotate(shape, 45)
    fig, ax = plt.subplots()
    plot(ax, r, 'blue', 'Original') plot(ax, r, 'green'
'rotate')
    ax.set_title('2D rotation')
    plt.show()
```

OUTPUT:-

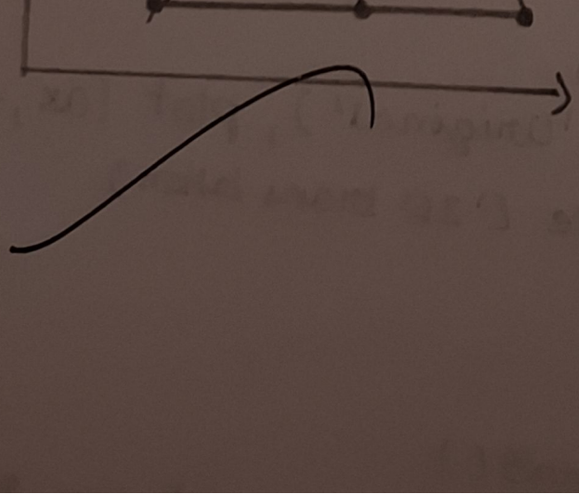
2D Translation



2D rotation :



2D Scaling



iii) S = Scale (Shape, 2, 0, 5)

```
fig, ax = plt.subplots()
plot(ax, shape, 'blue', 'original') plot(S, 'purple'
'scale').
ax.set_title('2D Scaling')
plt.show()
```

RESULT:

The program to draw the triangle for the basic transformation in python.

14/8/26

BASIC TRANSFORMATIONS FOR 2D SHAPES FOR RECTANGLE

Exp 9

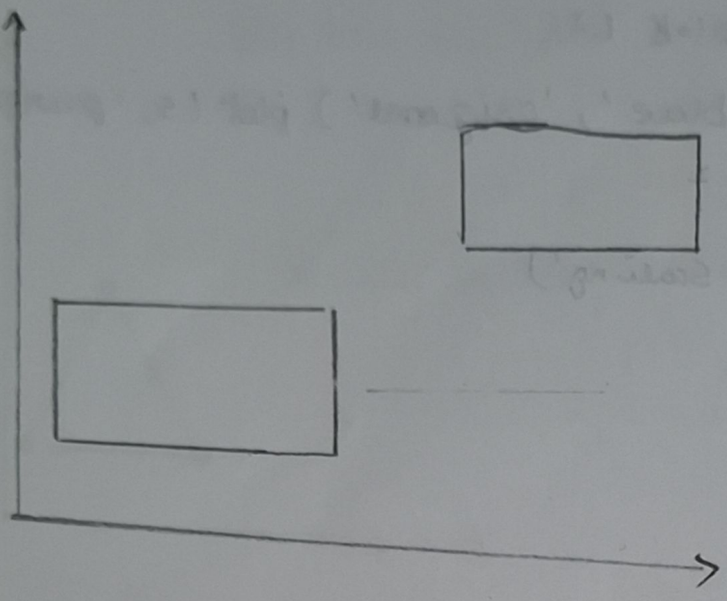
AIM:

To implement the basic transformation for 2D shapes to draw the rectangle in python.

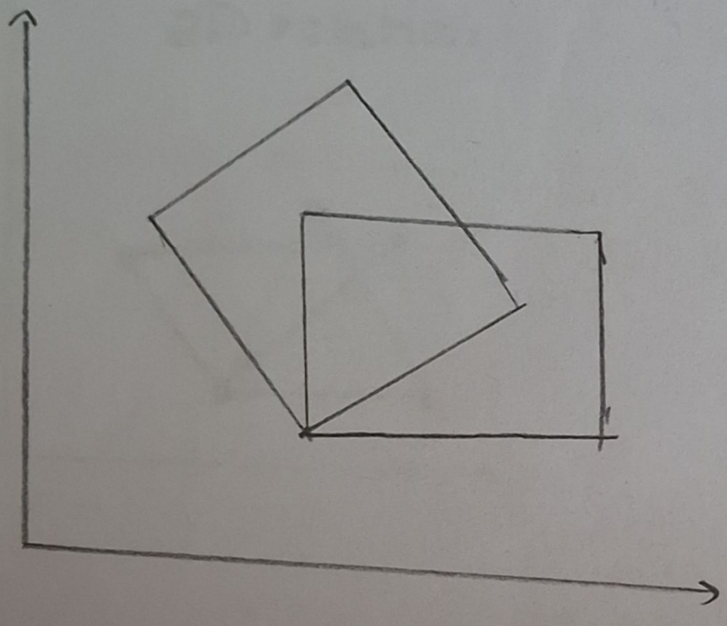
PROGRAM:

```
import numpy as np
import matplotlit.pyplot as plt
def translate (s, dx, dy):
    return s + [dx, dy]
def rotate (s, angle):
    a=np.radiance (angle)
    R = [[ np. (os (a) -np. sin (a)], [np. Sin(a).
np. (os [a)] ]
    return s@ np.array (R).T

def scale (s, sx, sy):
    return s = [sx, sy)
def plot (ax, s, color, label):
    ax, plot (s [:, 0], s [:, 1], color = color.
label = (abel)

def
t= translate (rect, 1, 2.5)
fig, ax = plt.subplots

plot (ax, rect, 'blue', 'original'), plot
(ax, t, 'red', 'Translation')
ax.set_title ('2D Translation')
```

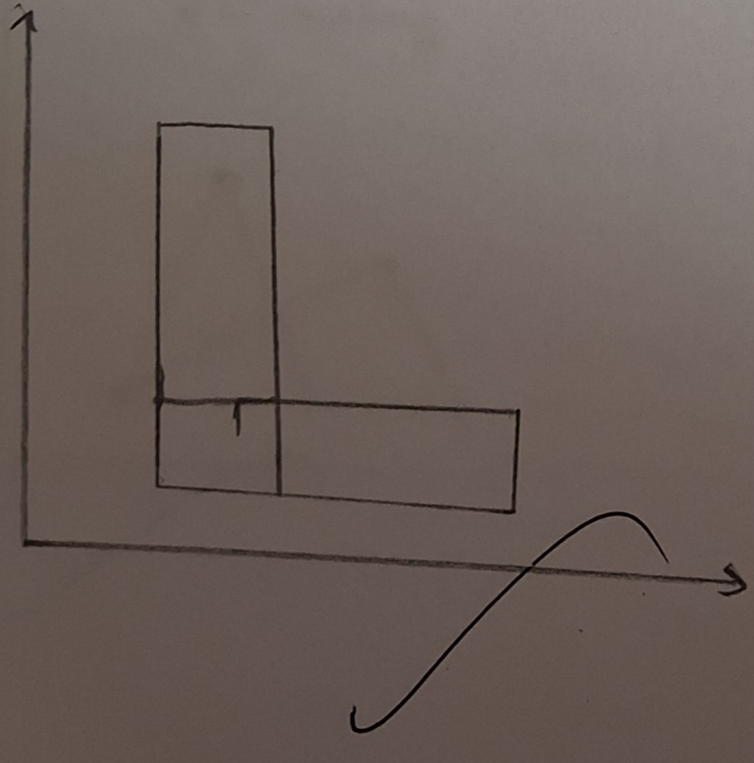
OUTPUT



2D Translation



2D Rotation



2D Scaling

```
ax.grid() ax.axis('equal') ax.legend()
plt.show()

r= rotate(rect, 30)
fig, ax = plt.subplots()
plot(ax, rect, 'blue', 'original'), plot
(ax, r, 'green', 'Rotation')
ax.set_title('2D rotation')
ax.grid() axaxis('equal') ax.legend()
plt.show()

S = scale(rect, 0.5, 1.5)
fig, ax = plt.subplots()
plot(ax, rect, 'blue', 'original')
plot.show()
```

RESULT:

The program to draw the rectangle for the basic transformation in python.

Exp 10

21/8/26

SUTHERLAND - HODGMAN POLYGON CLIPPING ALGORITHM

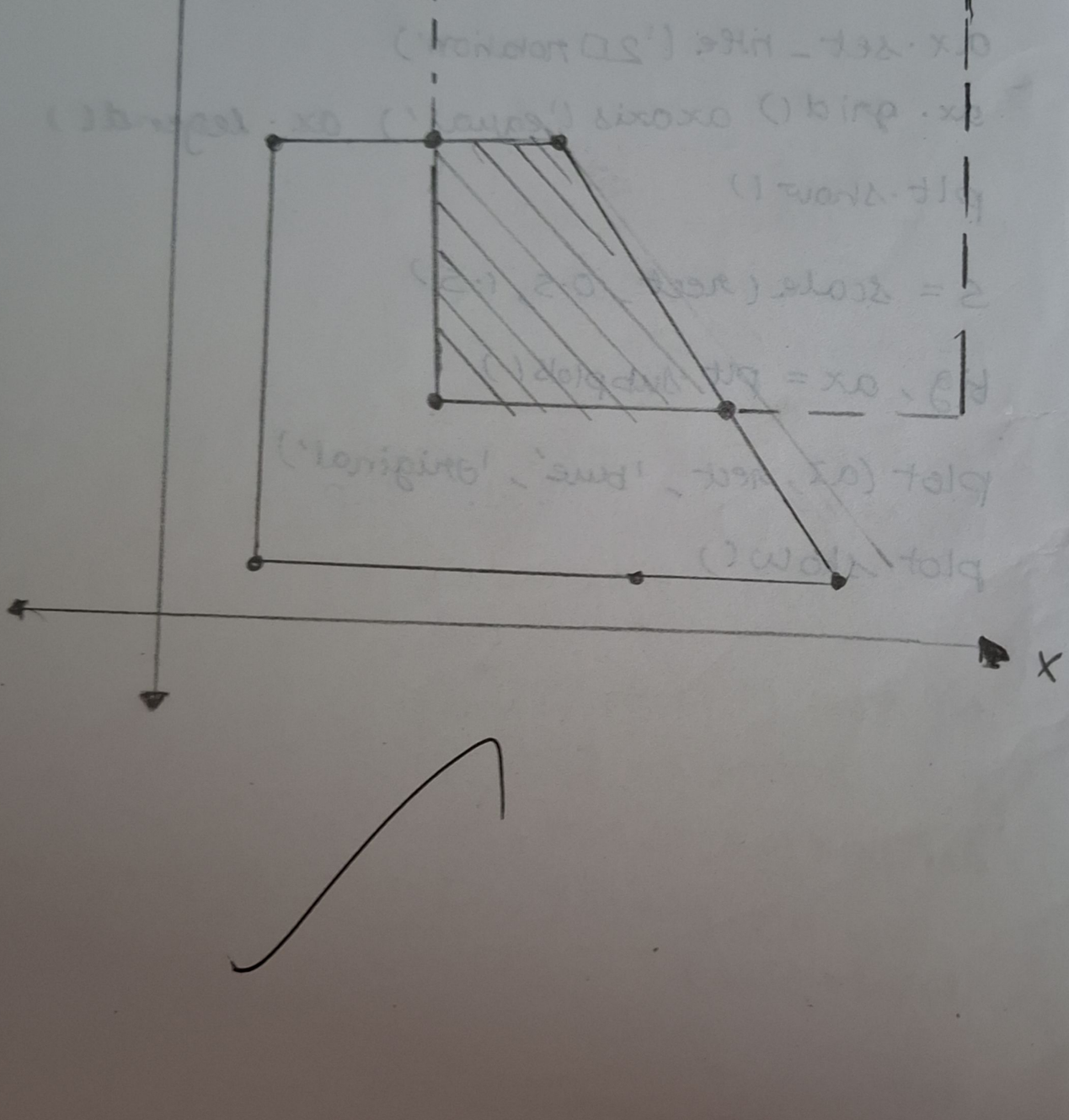
AIM:

To implement the algorithm to clip a subject polygon against a rectangular clipping window using python.

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
def inside(p, a, b):
    return np.cross(b-a, p-a) >= 0
def intersect(p1, p2, a, b):
    d1 = p2 - p1
    d2 = b - a
    c = np.cross(a - p1, d2) / np.cross(d1, d2)
return p2 + c * d1
def clip(subject, window):
    output = list(subject)
for i in range(len(window)) - 1:
    a, b = window[i], window[i+1]
    inp, output = output, []
for j in range(len(inp)):
    p1, p2 = inp[j-1], inp[j]
    in1, in2 = inside(p1, a, b) inside(p2, a, b)
if    in2:
    if not in1:
        output.append(intersect(p1, p2, a, b))
elif in1
    output.append(intersect(p1, p2, a, b))
return np.array(output)
subject = np.array([[0.5, 0.5], [3, 0.5], [2, 3]
[0.5, 3], [0.5, 0.5]])
```

OUTPUT:



```
window = np.array ([[1,1],[4,1],[4,4]
[1,4],[1,1]])

result = clip (subject, window)
plt.plot (subject [i,0], subject [:,1], 'b-o', label
='subject')
if len (result) >0:
        result = np.vstack ([result, result [0]])
plt.title ('polygon clipping'), plt.xlable ('x'), plt.ylabel
('y')
plt.legt()
plt.show()
```

RESULT:

The program was for the sutherland-hodgman polygon clipping algorithm is done.

Exp 11

11/9/26

3D SHAPES TRANSFORMATION FOR CUBE

AIM:

To implement the transformation for 3D shapes for cube using python.

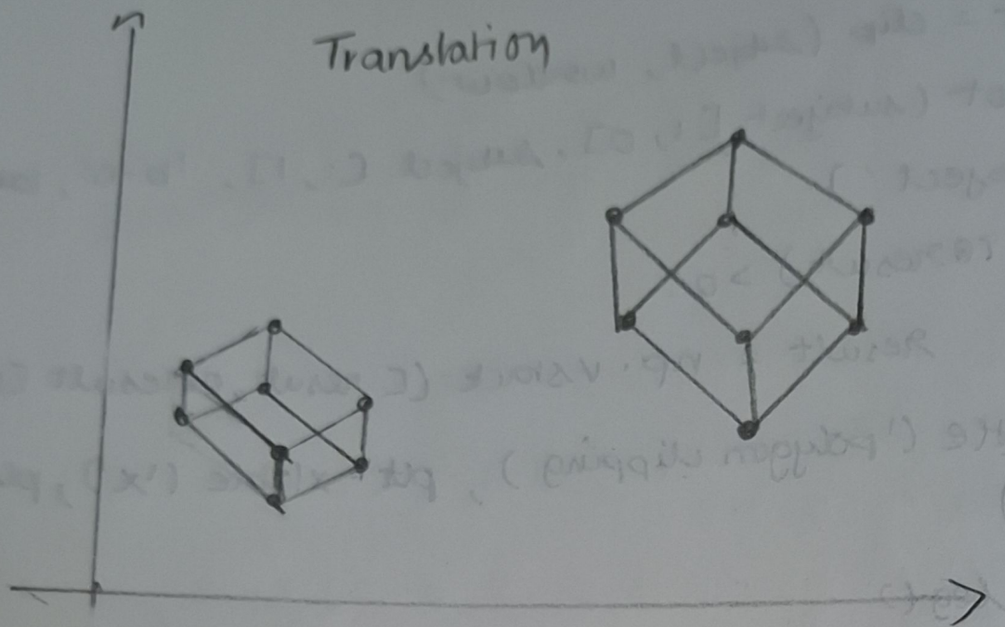
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3D, art3D, import ploy3D
V= np.array [[0,0,0], [1,0,0], [0,1,0], [0,0,1], [1,1,0]
[1,0,1] [0,1,1], [1,1,1] ])
H=np.c_ [V, np.ones (8)]
F =[[0,1,4,2], [3,5,7,6] [0,1,5,3] [2,7,7,6] [0,2,6,3]
  [1,4,7,5]]
def plot (T, title, color):
    w = (T@M.T, T [:, :3]
        fig = plt.figure (figsize = (8 6))
        ax = fig.add_subplot (111, projection = '3d')
  for v,c, in [(V,'blue') (N, color)]
        for i,j in E:
            ax.plot (xip[V[i] [i]) color= c)
            ax.scatter (V[i, 0], V[:, 1], V[:2], color=c)
        faces = [[V[k] for k in f] for f in f]
        plt.show()

T= np.array ([1,0,0,02], [0,1,0,2], [0,0,1,2]
    [0,0,0,1]])
a=np.pi/4
T= np.diag ([1,5,0,5,2,1] plot [t]
```

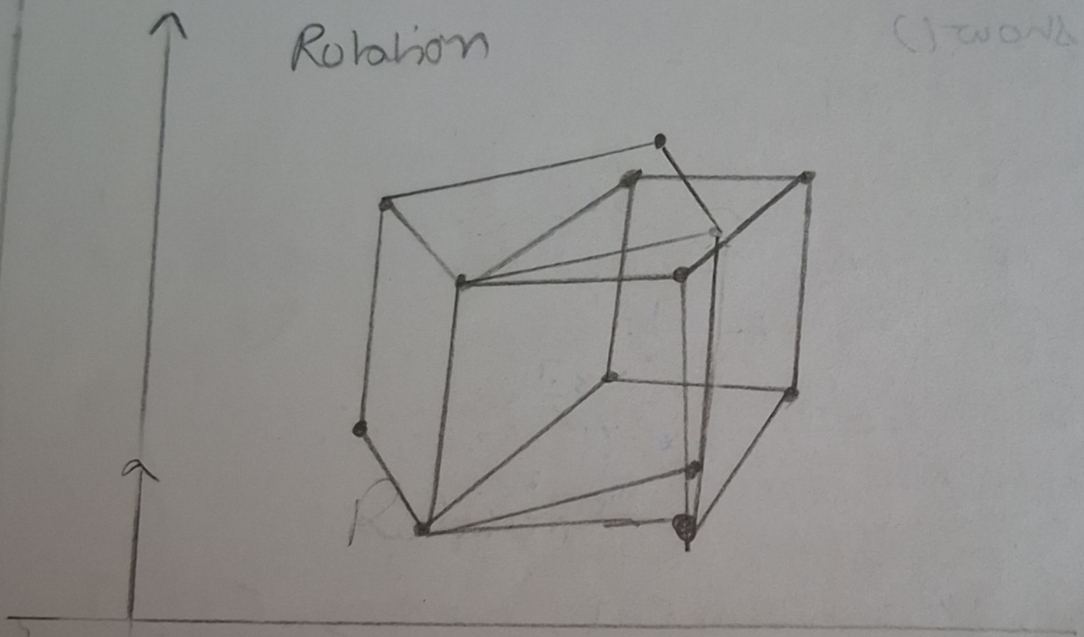
RESULT:

The program for 3D transformations for cube is done.

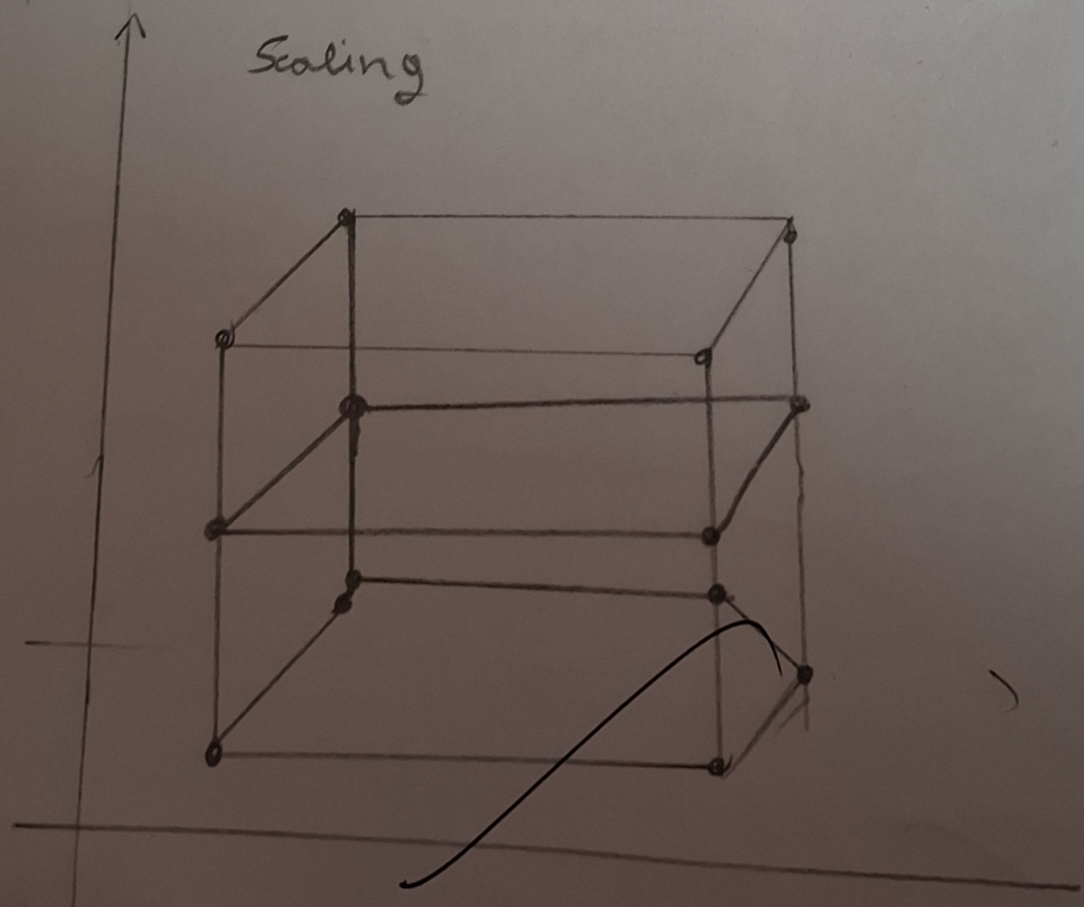
Translation



Rotation



Scaling



Exp 12

18/9/26

3D TRANSFORMATIONS FOR CONE

AIM:

To implement the program for 3D shapes transformation for cone.

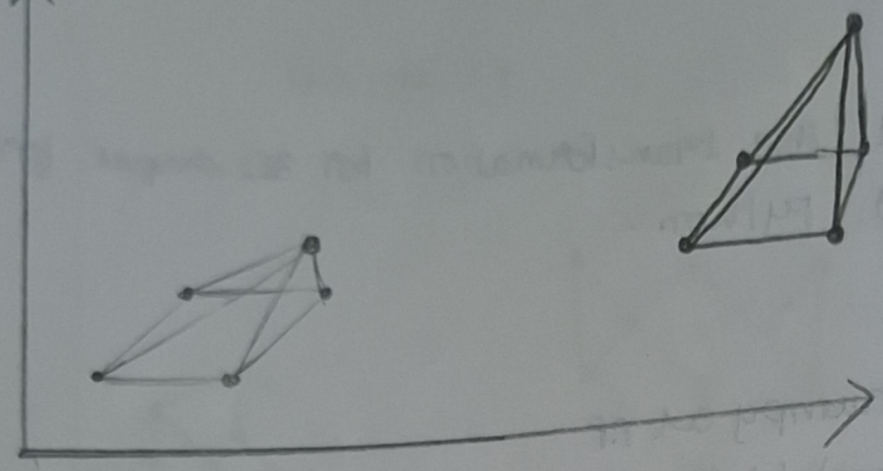
PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits, mplot3d.art3D import poly3Dcollection
V = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[5,5,1]])
H = np.c_[V, np.ones(5)]
E = [(0,1),(1,2),(2,3),(3,0),(0,4),(1,4),(2,4),(3,4)]
F = [[0,1,2,3],[0,1,4],[1,2,4],[2,3,4],[2,3,4]
[3,0,4]]
def plot(T, title, color):
        w = (T @ H.T).T[:,:3]
        fig = plt.figure(figsize=(8,6))
        ax = fig.add_subplot(111, projection '3d')
.for P,C in [V, 'blue'),(W, color)]:
            for i,j in E:
            ax, add_collections3d(poly3Dcollection
([[P[k] for k in f] for f in F] alpha = 2 facecolor
= C))
ax.set[xlabel='x', ylabel='y', zlabel='z', title=title)
plot.show()
T = np.diag([1.5, .5, 2, 1])
```

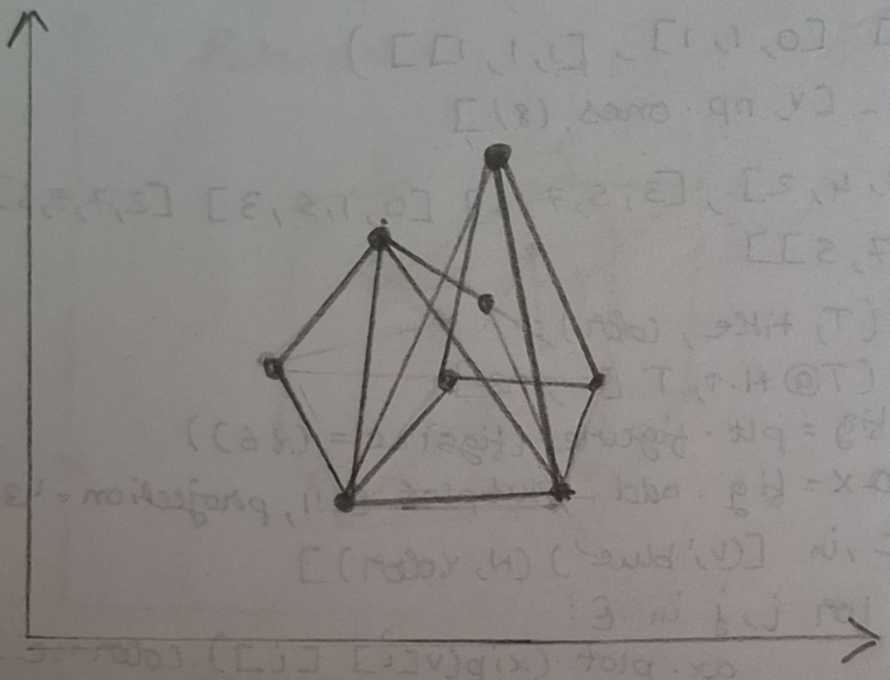
RESULT:

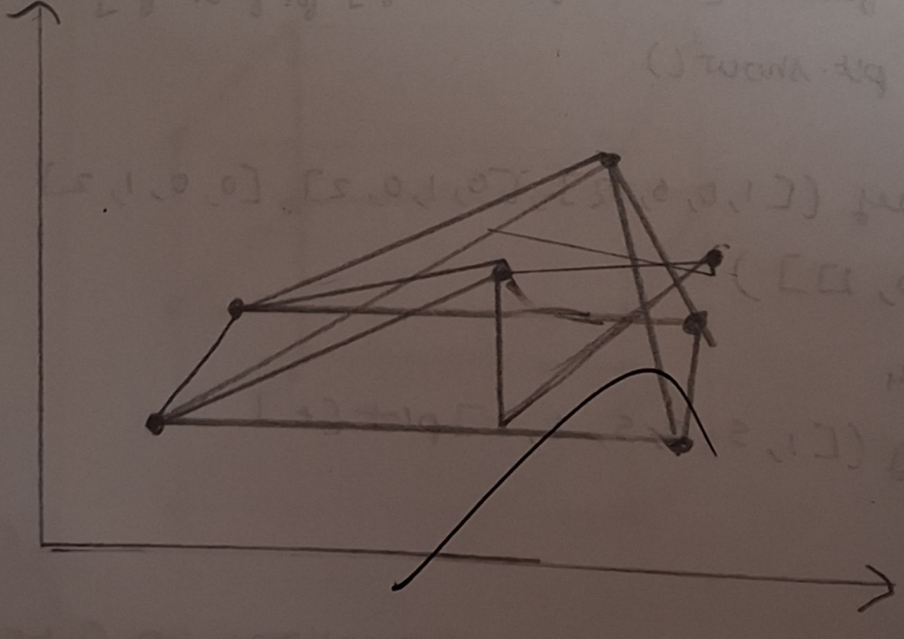
The program for 3D transformation for cone is done implemented.

OUTPUT



TRANSLATION





Exp 13

25/9/26 RENDERING 3D OBJECTS USING POLYGONS AND SHADING TECHNIQUES

AIM:
To create and render 3D objects using polygonal modelling and apply flat shading basic color techniques.

PROGRAM:

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
import numpy as np
vertices = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0]
  [0,0,1] [1,0,1] [1,1,1] [0,1,1])
faces =[[vertices[j] for j in [0,1,2,3]],
    [vertices[j] for j in [4,5,6,7]]
    [vertices[j] for j in [0,1,5,4]]
colors=['red', 'blue', 'green','yellow','orange','purple']
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_zlabel('z')
ax.set_xlim(0,1)
ax.set_ylim(0,1)
plt.show()
```

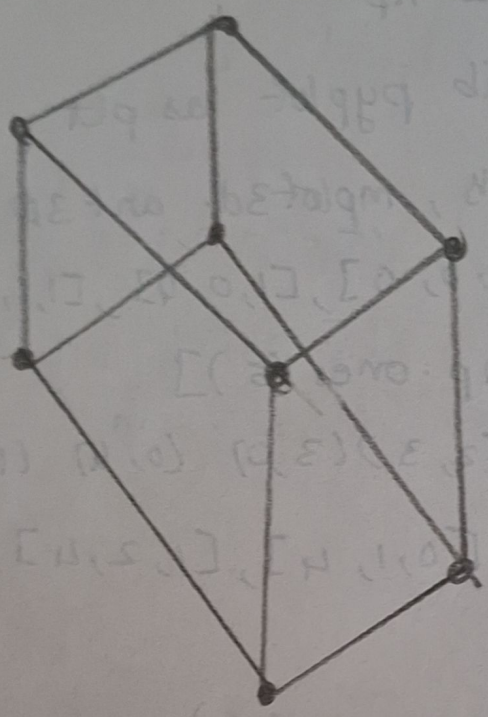
RESULT:
3D polygonal objects were successfully created and rendered.

Exp 13

25/9/26

AIM:
To
m
4

PROG



EXP 14

6/10/26

MULTIMEDIA APPLICATION INTERFACES IMAGES SOUND and VIDE IN UI

AIM:

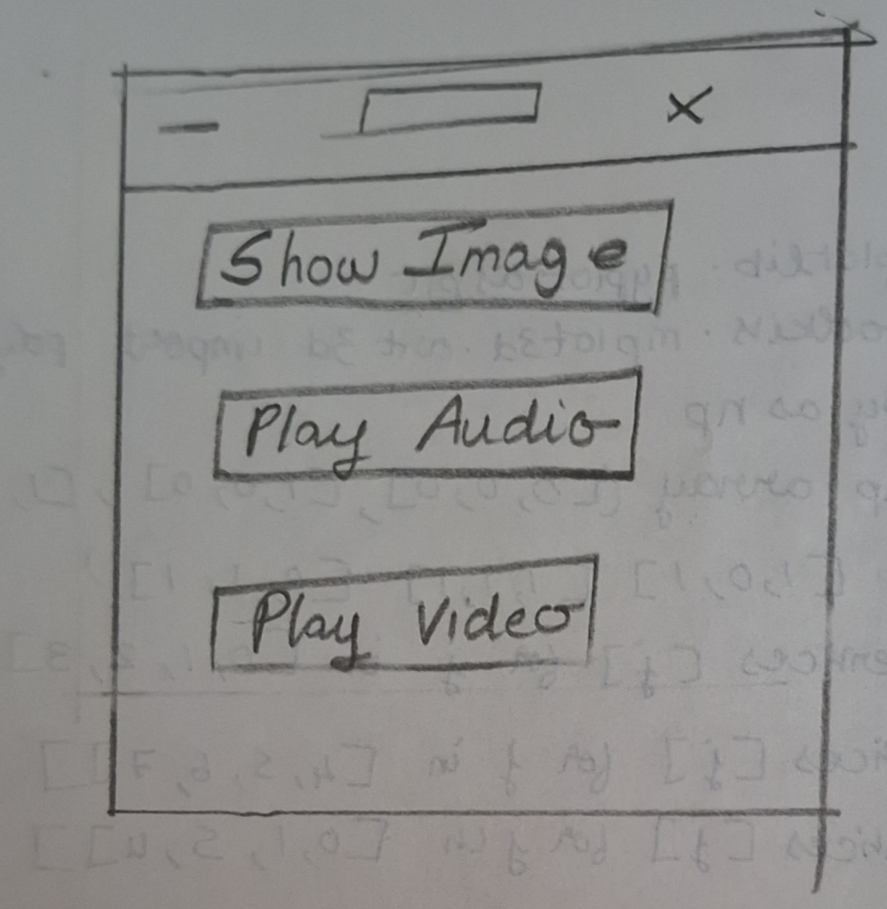
To develop a simple multimedia application that intergrates images, vides and audio in UI.

PROGRAM:

```
import tkinter as tk
from Pss import image, imagetk
from playground import playground
import v2
import threading
def image() {
    img = image.open("Simple_img.jpg") resize (300, 300)
    photo = ImageTk.photoImage(imp)
    panel.config (image= photo)
    palel.image = phote
}
def video(){
    cap = cv2.videocapture ("Sample.mp4)
    while cap.isOpened():
        ret.f = cop.oread()
        if not ret:
            break
        cv2.inshow ("video", frame)
cap.release()
cv2.destroy All Windows()
.out.title ("Multimedia App")
panel.pack()
def audio()
    playgroud ("Sample_audio.mp3")
proof.mainloop()
```

RESULT:

A multimedia application was successfully created and intergrate images, video audio.


Show Image
Play Audio
Play Video

Exp 15

al10126 PVH

CAPTURE VIDEO/AUDIO FROM WEBCAM OR MICROPHONE AND DISPLAY MULTIMEDIA

AIM:

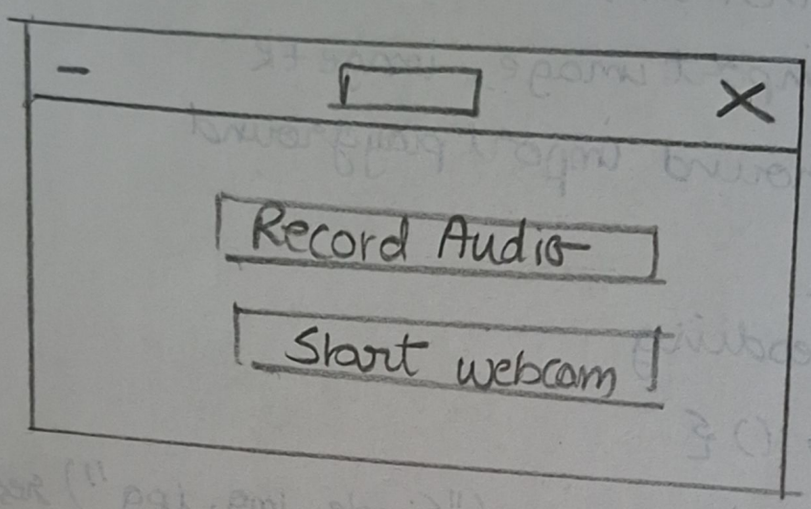
To develop a program that capture live video, audio from the webcam and microphones

PROGRAM:

```
import tkinter as tk
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import tmpading
def audio():
    fs = 44100
    print ('Recording ...')
    data = sd.rec (5* fs, samplerate = fs, channels = 2)
    sd.wait()
    write ("recorder_audio.wav", fs, data)
    print ("Audio Saved")
def video():
    cap = cv2.VideoCapture(0)
    while True:
        ret, fram = cap.read
    if not ret:
        break
if cv2.waitKey (1) & 0xFF = ord('q'):
  break
cap.release()
cv2.destroyAllWindows()
root.title ("Audio Video Capture")
root.mainloop()
```

RESULT:

Live video and Audio were successfully Captured using webcam microphones.


Record Audio
Start webcam